examenPractico

April 7, 2026

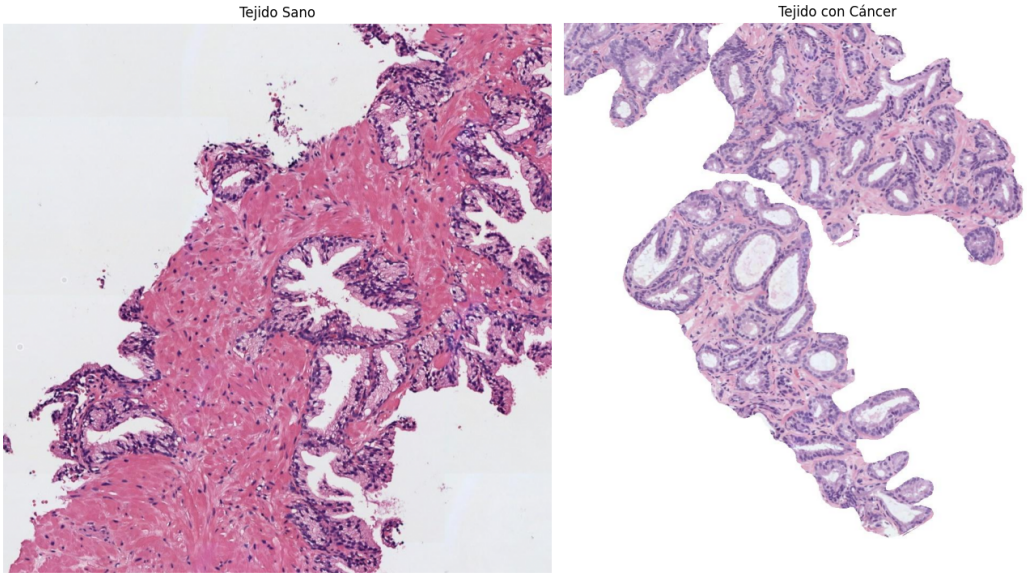
0.1 Examen Practico - Segundo Parcial

0.2 Nombre: *Adriel Bedoya*

0) Cargar una de las imágenes histológicas

[78]:
```
# Utilizar la librería skimage.io para leer la imagen 'histo_x.jpg' en formato␣
↪RGB.
# Normalizar la imagen para que los píxeles se encuentren en el rango [0, 1]

import matplotlib.pyplot as plt #para las gráficas
import numpy as np #para las operaciones
```

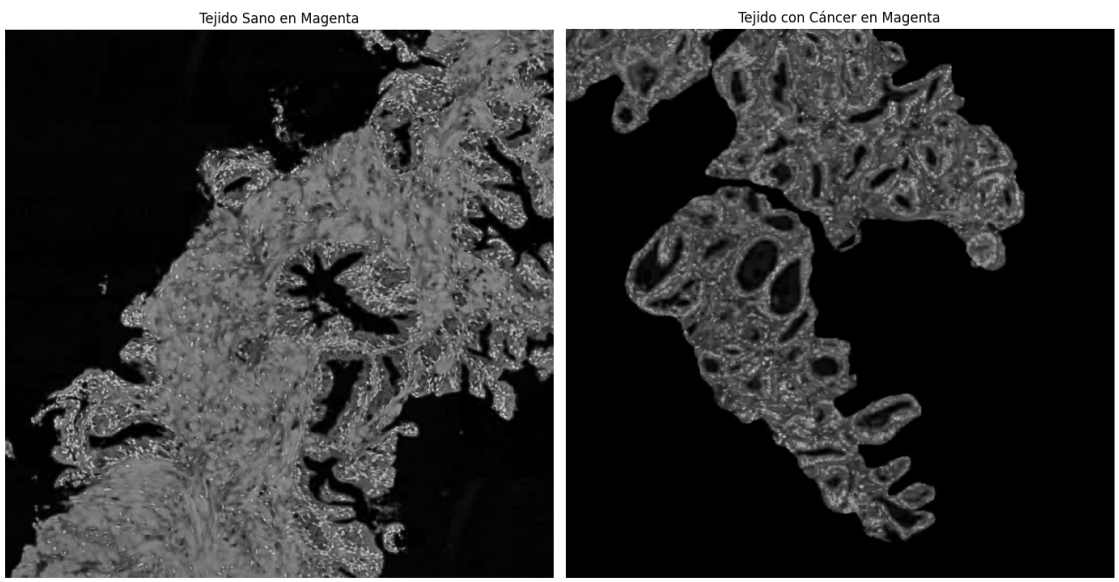
```python
from skimage import io #para ver las imágenes
from skimage.filters import gaussian, threshold_otsu #Para generar la máscara␣
↪del lumen, aplicar filtro gausianno y umbralizar automáticamente
from skimage.morphology import remove_small_objects #Para limpiar la máscara␣
↪que hicimos en lo de arriba (limpiar los puntitos)
from skimage.segmentation import flood #para ver los bordes y llenarlos, luego␣
↪aplicaré una capa inversa xd
from scipy.ndimage import binary_fill_holes #Rellenar los huecos
from skimage.measure import find_contours #Para los contornos
from skimage.measure import label, regionprops #label para asignar un número␣
↪único a cada island y el regionprops para calcular el área, centroide,␣
↪perímetro y bbox
import polars as polarsote #para el dataframe, el reemplazo de pandas

try:
    #1) Cargamos las imágenes
    imagen_1 = io.imread('histo_1.jpg') # Celula sana
    imagen_2 = io.imread("histo_2.jpg") # Célula con cáncer

    #2) Normalizamos las imágenes
    #Para esto, debemos dividir para 255 (estandar para imágenes de 8 bits)
    imagen_1_normalizada = imagen_1/255
    imagen_2_normalizada = imagen_2/255

    #3) Visualizamos la gráfica de las imágenes
    plt.figure(figsize=(14,8))

    # 3.1) Imágen sin cáncer
    plt.subplot(1,2,1)
    plt.imshow(imagen_1_normalizada)
    plt.title("Tejido Sano")
    plt.axis("off") #Con eso no salen números en la gráfica, no los necesito

    #3.2) Imagen con cancer

    plt.subplot(1,2,2)
    plt.imshow(imagen_2_normalizada)
    plt.title("Tejido con Cáncer")
    plt.axis("off") #Con eso no salen números en la gráfica, no los necesito

    #3.3) Para ajustar el espacio entre imágenes usaremos el tight_layout
    plt.tight_layout()
    plt.show()

except FileNotFoundError:
    print("No se encontraron las imágenes")
```



1) Realizar una transformación de color para convertir la imagen al espacio de color CMYK

[79]:
```
# Extraer la componente magenta de la imagen (que corresponde a la región␣
↪tisular)

def extraer_color(img_color_full):
    #Vamos a convertir una imagen rgb que ya está normalizada a CMYK y devolver␣
↪el canal del Magenta

    #1) Vamos a extraer los demás canales
    G = img_color_full [:,:,1]

    # 1.2) Calculamos el canal negro
    K = 1-np.max(img_color_full, axis = 2)

    # 1.3) Calculamos el magenta con epsilion para que la división no se haga *␣
↪0
    epsilon = 1e-8
    M = (1 - G - K) / (1 - K + epsilon)

    # 1.4) Limpiamos los valores que estén fuera de ese rango con np.clip
    return np.clip(M,0,1)


#2) Extraemos los componentes
imagen_magenta_1 = extraer_color(imagen_1_normalizada)
imagen_magenta_2 = extraer_color(imagen_2_normalizada)
```

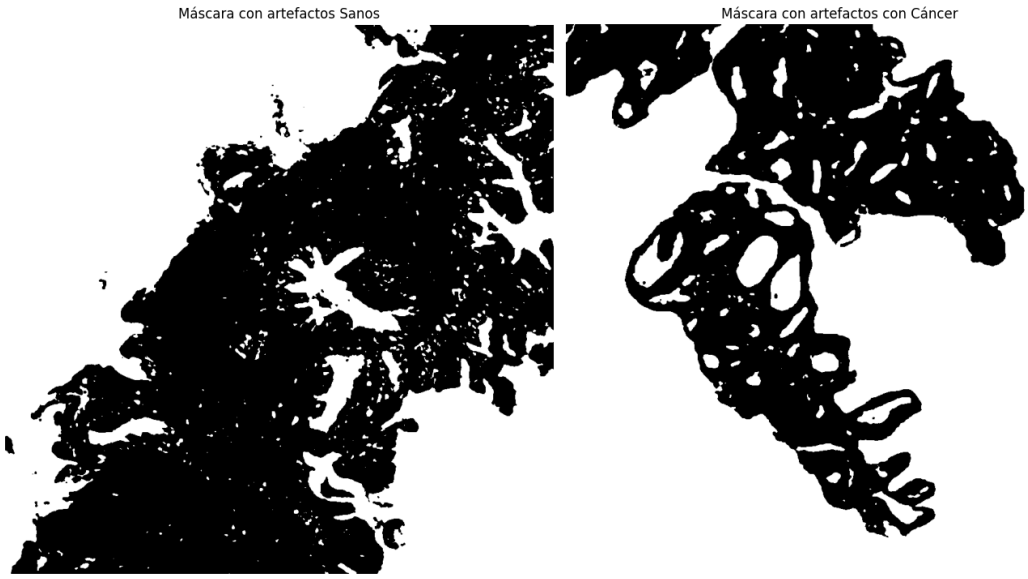
```
# 3) Visualizar la imagen del canal magenta

plt.figure(figsize=(14, 8))

# 3.1) Imagen sin cáncer
plt.subplot(1, 2, 1)
plt.imshow(imagen_magenta_1, cmap='gray')
plt.title("Tejido Sano en Magenta")
plt.axis('off')

# 3.2) Imagen con cancer
plt.subplot(1, 2, 2)
plt.imshow(imagen_magenta_2, cmap='gray')
plt.title("Tejido con Cáncer en Magenta")
plt.axis('off')

plt.tight_layout()
plt.show()
```



2) Umbralizar la imagen para separar los píxeles del fondo de la región tisular

[80]:
```
# Aplicar un filtro gaussiano de tamaño 5x5 y después utilizar el método de␣
↪Otsu de manera que los píxeles correspondientes al lumen y al background de␣
↪la imagen sean 1s y el resto de los píxeles tengan un valor de 0.
# Nota: Recordar que el método de Otsu requiere como input una imagen en el␣
↪rango [0-255] en formato "uint8".
```

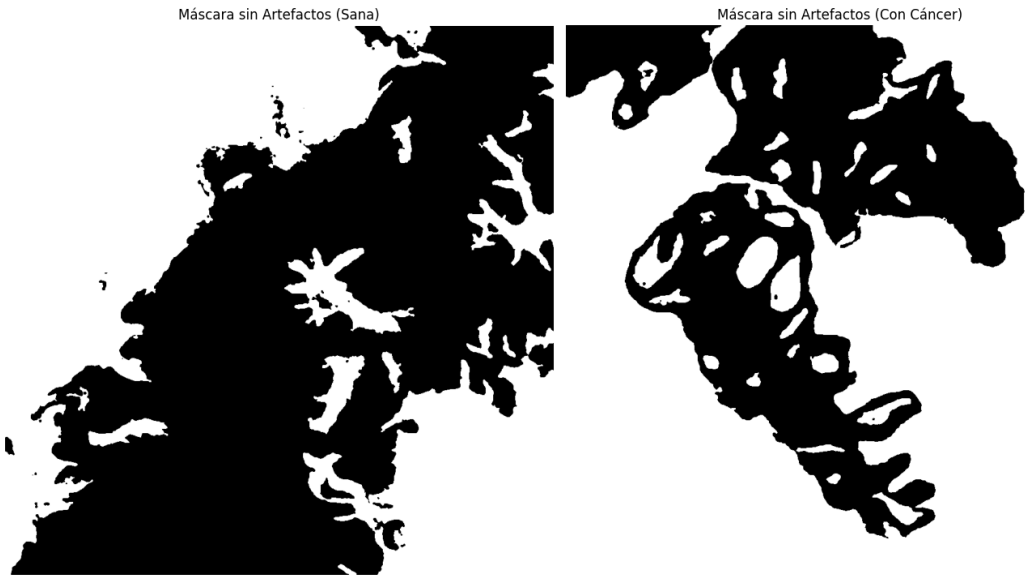
```python
#1) Generamos una función para poder hacer las máscaras del lumen (para no␣
↪estar repitiendo el code xd)
#Gracias al filtro gaussiano tendremos un suavizado, luego vamos a convertir a␣
↪uint8 y generaremos una máscara binaria donde el lumen es 1 y el tejido es 0
def mascara_lumen(imagen_magenta):

    #1) Filtro Gauss (sigma debe ser 1 para simular un kernel de 5x5)
    img_gauss = gaussian(imagen_magenta, sigma = 1)

    #2) Escalar a 0-255 y convertir a uint8 para luego umbralizar con Otsu
    img_con_uint8 = (img_gauss * 255).astype(np.uint8)

    #3) Hacemos que Otsu calcule el umbral automáticamente
    thresh = threshold_otsu(img_con_uint8)

    #4) Creamos la máscara : Si el lumen es menor al umbral, entonces será uno.␣
↪Por otro lado, si el tejido es mayor al umbral, entonces será 0
    return (img_con_uint8 < thresh).astype(np.uint8)


primera_mascara = mascara_lumen(imagen_magenta_1)
segunda_mascara = mascara_lumen(imagen_magenta_2)


# Visualizar la máscara resultante

fig, axes = plt.subplots(1, 2, figsize=(14, 8))

axes[0].imshow(primera_mascara, cmap='gray')
axes[0].set_title("Máscara con artefactos Sanos")
axes[0].axis('off')

axes[1].imshow(segunda_mascara, cmap='gray')
axes[1].set_title("Máscara con artefactos con Cáncer")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



3) Limpiar la imagen eliminando los artefactos de lumen (objetos blancos pequeños que no son lúmenes)

[81]:
```
# Utilizar la librería skimage.morphology.remove_small_objects para eliminar␣
↪aquellos objetos cuya área sea menor a 300 píxeles
# Más información en https://scikit-image.org/docs/dev/api/skimage.morphology.
↪html#skimage.morphology.remove_small_objects

# 1) Limpiar la imágen borrando los objetos que sean < a 300 px

# 1.1) Imagen sin Cáncer

primera_mascara_limpia = remove_small_objects(primera_mascara.astype(bool),␣
↪max_size = 290) #aqui cambie, es 300

# 1.2) Imagen con Cáncer

segunda_mascara_limpia = remove_small_objects(segunda_mascara.astype(bool),␣
↪max_size = 290) #aqui cambie, es 300

# 2) Visualizaer la máscara resultante
plt.figure(figsize=(14, 8))
plt.subplot(1, 2, 1)
#Usamos .astype(float) o int para que plt.imshow no tenga problemas al graficar
plt.imshow(primera_mascara_limpia.astype(float), cmap='gray')
plt.title("Máscara sin Artefactos (Sana)")
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(segunda_mascara_limpia.astype(float), cmap='gray')
plt.title("Máscara sin Artefactos (Con Cáncer)")
plt.axis('off')

plt.tight_layout()
plt.show()
```

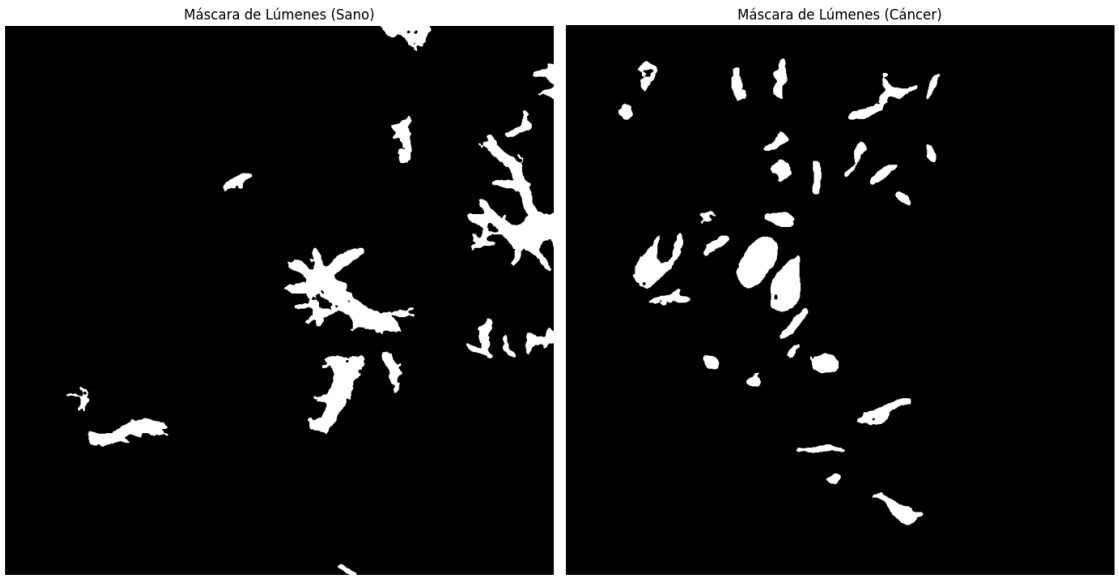


4) Rellenar con 0s el fondo de la imagen para quedarnos únicamente con los lúmenes

[82]:
```
# Aplicar el algoritmo de expansión a partir de semillas (region growing) de␣
↪manera que únicamente los lúmenes sean blancos
# y el resto de la imagen negra. Pista: utilizar dos semillas. Nota: Se pueden␣
↪fijar las semillas de manera manual, pero
# se valorará positivamente a aquell@s que desarrollen una función para␣
↪encontrarlas automáticamente.

"""
En lugar de intentar adivinar dónde está cada lumen (que pueden ser muchos y␣
↪estar en cualquier lado), voy a identificar mejor el fondo. Usaré semillas␣
↪en las esquinas asumiendo
que el fondo siempre toca las esquinas de la imagen. Al lanzar el algoritmo␣
↪flood (para el llenado), haré como un pintado en todo el espacio en blanco␣
↪exterior. Después, el algoritmo
se expande como una mancha de aceite desde las 4 semillas que pondré, pero se␣
↪va a detener en cuanto choque con el tejido (cuando la máscara sea 0).
```

```
Algo importante es hacer lógica inversa al tomar la máscara original (la que␣
↪tenía lúmenes y el fondo) y le quito (con ~ o NOT) lo que identifique como␣
↪fondo. Lo que resta simplemente son los lúmenes
internos

"""

def obtener_lumenes(mascara):

    #1) Sácamos la máscara en modo boolean para hacer las operaciones
    mascara_boolean = mascara.astype(bool)

    #2) Creamos una máscara para el fondo (al inicio será todo negro/falso)
    fondo = np.zeros_like(mascara_boolean, dtype=bool)

    #3) Haré las semillas de manera automática por esquina
    # (0,0) arriba-izquierda, (0,-1) arriba-derecha, (-1,0) abajo-izquierda,␣
↪(-1,-1) abajo-derecha

    alto, ancho = mascara_boolean.shape
    semillas = [(0, 0), (0, ancho - 1), (alto - 1, 0), (alto - 1, ancho - 1)]

    for semilla in semillas:
        #el fllod encontará todos los píxeles conectados a la semilla que␣
↪tengan el mismo valor
        #el conectivity = 1 es para que solo mire arriba, abajo, izquierda y␣
↪derecha
        area_encontrada = flood(mascara_boolean, seed_point = semilla,␣
↪connectivity = 1)
        fondo = fondo | area_encontrada

    lumen = mascara_boolean & (~fondo)

    #4) Vamos a convertir a 0 y 1 para que sea más fácil calcular en otros pasos
    return lumen.astype(np.uint8)

lumen_imagen1 = obtener_lumenes(primera_mascara_limpia)
lumen_imagen2 = obtener_lumenes(segunda_mascara_limpia)



# 5) Visualizar la máscara resultante.
fig, axes = plt.subplots(1, 2, figsize=(14, 8))

axes[0].imshow(lumen_imagen1, cmap='gray')
axes[0].set_title("Máscara de Lúmenes (Sano)")
axes[0].axis('off')
```

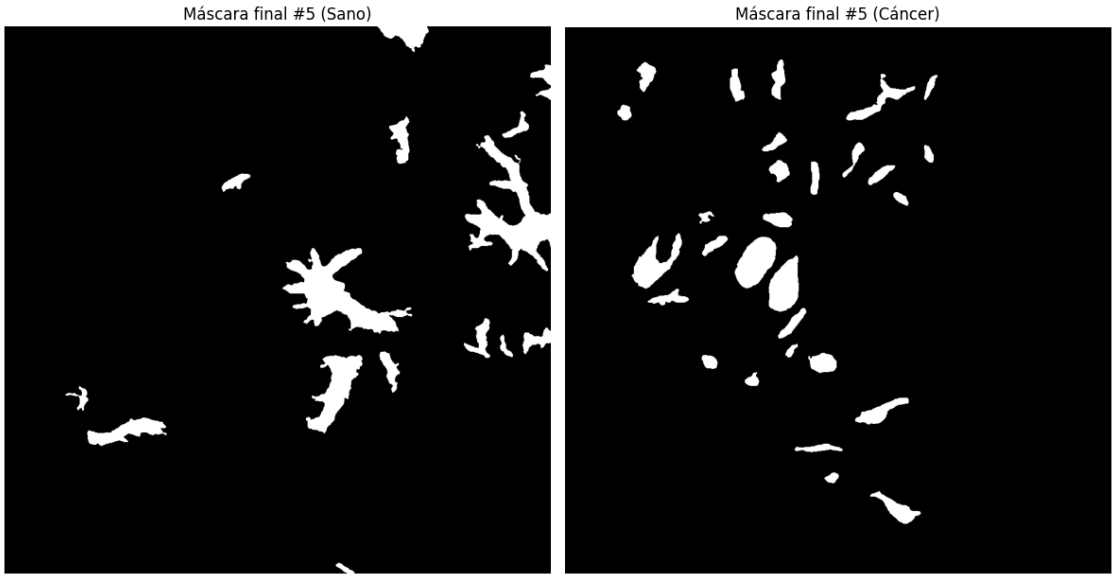
```
axes[1].imshow(lumen_imagen2, cmap='gray')
axes[1].set_title("Máscara de Lúmenes (Cáncer)")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



5) Rellenar los objetos de los lúmenes

[83]:
```
#1) Rellenar los lúmenes con la función binary_fill_holes de la librería scipy.
↪ndimage.morphology
#Eliminamos el ruido negro con esta opción, dando una mejor vista de los lúmenes

lumen1_rellenado = binary_fill_holes(lumen_imagen1)
lumen2_rellenado = binary_fill_holes(lumen_imagen2)

#2) Visualizar la máscara resultante
fig, axes = plt.subplots(1, 2, figsize=(12, 8))

# 2.1) Imagen sin cáncer
axes[0].imshow(lumen1_rellenado, cmap='gray')
axes[0].set_title("Máscara final #5 (Sano)")
axes[0].axis('off')

# 2.2) Imagen con cáncer
axes[1].imshow(lumen2_rellenado, cmap='gray')
axes[1].set_title("Máscara final #5 (Cáncer)")
```

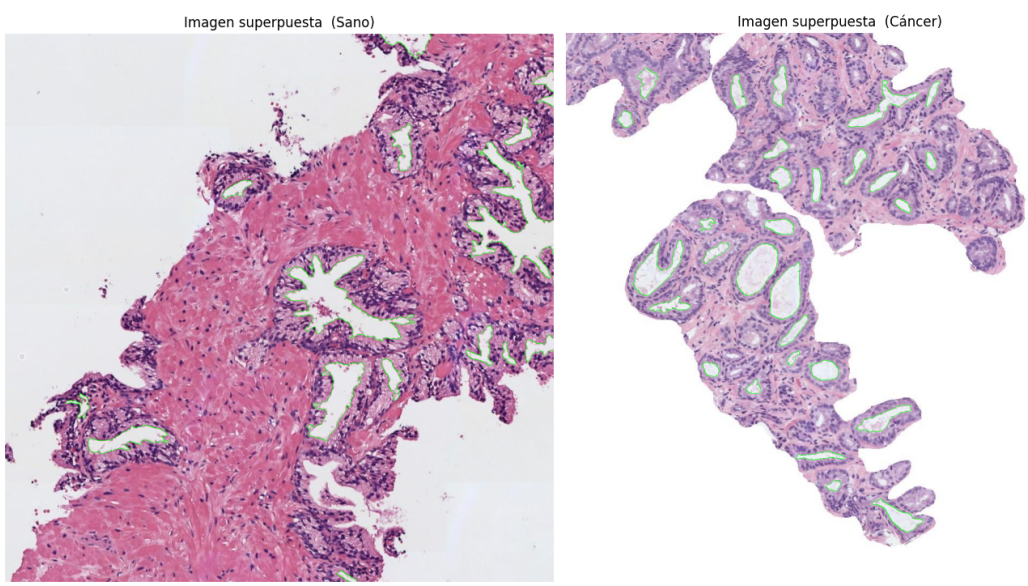
```
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



6) Detectar y dibujar los contornos de los lúmenes sobre la imagen original

[84]:
```
# Dibujar los contornos de los lúmenes en color verde sobre la imagen original␣
↪RGB. Nota: Utilizar los flags necesarios para que los contornos en verde␣
↪sean perfectamente visibles.

# 1) Definiré una función para dibujar los contornos manipulando los px␣
↪directamente

def dibujar_contornos(imagen , mascara):

    #1.1) Creamos una copia para no modificar la imagen original
    img_salida = imagen.copy()

    #1.2) Encontramos contornos en la máscara binaria. Devolviendo una lista de␣
↪arrays (n,2) float
    contornos = find_contours(mascara, level = 0.5)

    #1.3) Iteramos sobre cada objeto detectado con un bucle for
    for contorno in contornos:
        #1.3.1) Convertimos las coordenadas float (fila y col) a índices enteros
        contorno = contorno.astype(int)
```

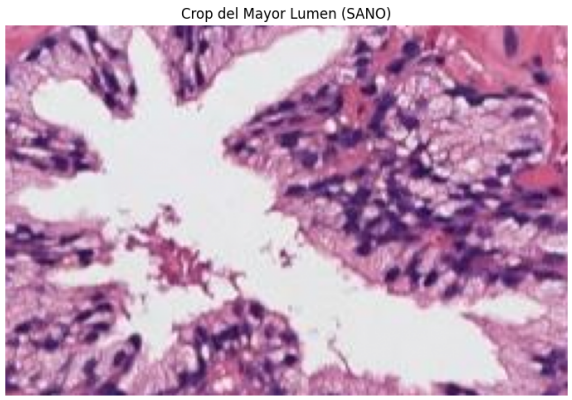
```
        #1.3.2) Dibujamos de color verde directamente en la matriz
        # usamos la indexación de numpy img[filas, col]
        img_salida[contorno[:,0], contorno[:,1]] = [0,1,0]

    return img_salida

imagen_con_contorno1 = dibujar_contornos(imagen_1_normalizada, lumen1_rellenado)
imagen_con_contorno2 = dibujar_contornos(imagen_2_normalizada, lumen2_rellenado)

# 2) Visualizar la imagen superpuesta
fig, axes = plt.subplots(1, 2, figsize=(14, 8))

# 2.1) Imagen 1 (Sana)
axes[0].imshow(imagen_con_contorno1)
axes[0].set_title("Imagen superpuesta  (Sano)")
axes[0].axis('off')

# 2.2) Imagen 2 (Cáncer)
axes[1].imshow(imagen_con_contorno2)
axes[1].set_title("Imagen superpuesta  (Cáncer)")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



7) Identificar y cropear el lumen más grande

[97]:
```python
import matplotlib.pyplot as plt
from skimage.measure import label, regionprops

# 1) Función para obtener el recorte del lumen más grande
def obtener_lumen_mamacoco(imagen, mascara):

    #1.1) Aseguramos que la máscara sea tratada como etiquetas
    etiquetado = label(mascara)
    regiones = regionprops(etiquetado)

    #1.2) Si no hay región para los lúmenes, entonces devolver la imágen␣
 ↪original
    if not regiones:
        print("Aviso: No se detectaron lúmenes")
        return imagen

    #1.3) Encontrar la región con el área más grande
    region_grande = max(regiones, key=lambda r: r.area)

    #1.4) Extraer las coordenadas del bounding box
    fila_inicial, columna_inicial, fila_final, columna_final = region_grande.
 ↪bbox

    #1.5) Realizar el recorte (Slicing)
    recorte = imagen[fila_inicial:fila_final, columna_inicial:columna_final]

    return recorte


recorte_imagen_1 = obtener_lumen_mamacoco(imagen_1_normalizada,␣
 ↪lumen1_rellenado)
recorte_imagen_2 = obtener_lumen_mamacoco(imagen_2_normalizada,␣
 ↪lumen2_rellenado)

# visualización
fig, axes = plt.subplots(1, 2, figsize=(14, 8))

# 2.1) Imagen sin cáncer
axes[0].imshow(recorte_imagen_1)
axes[0].set_title("Crop del Mayor Lumen (SANO)")
axes[0].axis("off")

# 2.2) Imagen con cáncer
axes[1].imshow(recorte_imagen_2)
axes[1].set_title("Crop del Mayor Lumen (CÁNCER)")
axes[1].axis("off")
```

```
plt.tight_layout()
plt.show()
```



8) Extraer 13 características geométricas que permitan caracterizar el lumen recortado

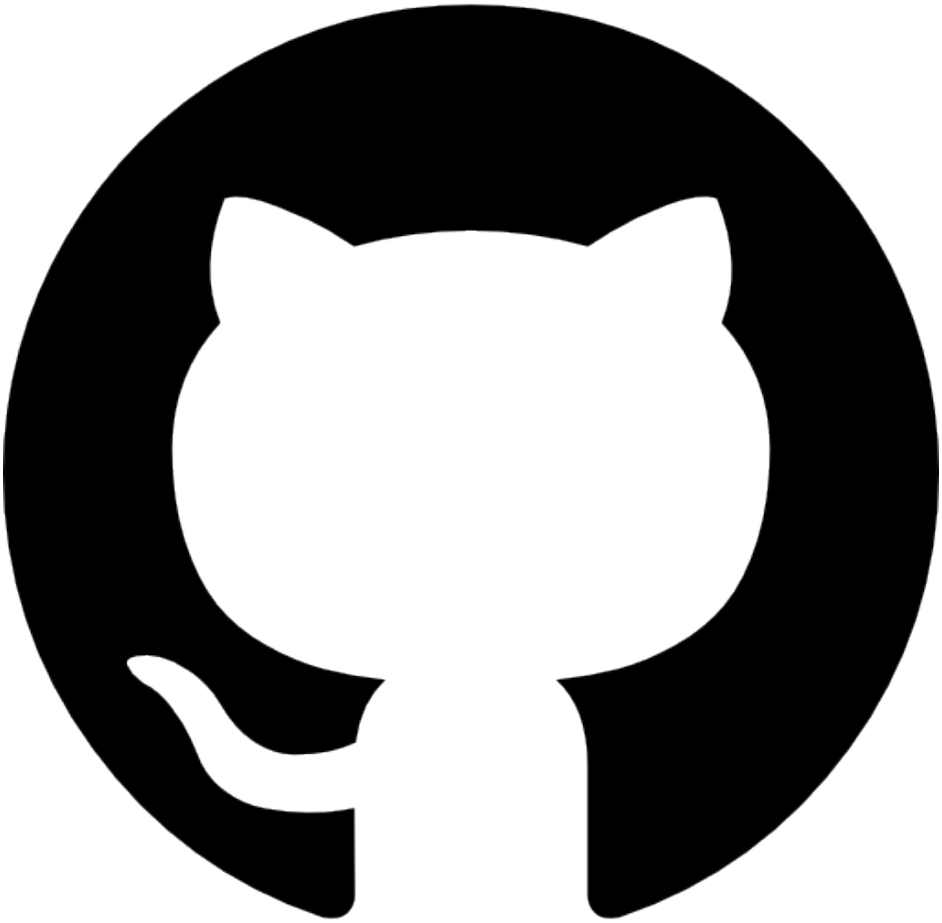
[102]:
```
#1. Calcular las siguientes características del crop del lumen de mayor área,␣
↪redondeando su valor hasta el cuarto decimal.

def extraer_caracteristicas(mascara, etiqueta):

    etiquetado = label(mascara)
    por_region = regionprops(etiquetado)

    #Si no hay región, devolver un none
    if not por_region:
        return None

    #1) Seleccionamos el lumen más grandote
    region = max(por_region, key= lambda r: r.area)

    # Las 13 características que se pidieron
    datos = {

        "Característica" : etiqueta,
        #1) Área
        "Área": float(region.area),
```

```
        #2) Bbox
        "Área BBox": float(region.area_bbox),
        #3) Área convexa
        "Área Convexa": float(region.area_convex),
        #4) Excentricidad
        "Excentricidad": float(region.eccentricity),
        #5) Diámetro Equivalente
        "Diámetro Equiv.": float(region.equivalent_diameter_area),
        #6) Extensión
        "Extensión": float(region.extent),
        #7) Diámetro Feret
        "Diámetro Feret": float(region.feret_diameter_max),
        #8) Eje mayor
        "Eje Mayor": float(region.axis_major_length),
        #9) Eje menor
        "Eje Menor": float(region.axis_minor_length),
        #10) Orientación
        "Orientación": float(region.orientation),
        #11) Perímetro
        "Perímetro": float(region.perimeter),
        #12) Solidez
        "Solidez": float(region.solidity),
        #13) Compacidad
        "Compacidad": float((region.perimeter**2) / (4 * np.pi * region.area))
    }

    return polarsote.DataFrame([datos])

# 1) Extraemos los datos de ambas imágenes
df_sano = extraer_caracteristicas(lumen1_rellenado, "Sano")
df_cancer = extraer_caracteristicas(lumen2_rellenado, "Cáncer")

# 2) Unir ambos dataframes con Polars
#usamos polarsote.concat para tener una tabla comparativa
df_finalizado = polarsote.concat([df_sano, df_cancer])

#3) Aplicamos el redondeo a 4 decimales
# Voy a usar una selección por tipo para no redondear la columna␣
↪"Características"

df_finalizado = df_finalizado.with_columns(polarsote.col(polarsote.Float64).
↪round(4))

display(df_finalizado)
```

```
shape: (2, 14)
```

```
Característica   Área      Área BBox   Área Convexa   …   Orientación   Perímetro␣
↪ Solidez   Compacidad
---              ---       ---         ---                ---           ---       ␣
↪ ---       ---
str              f64       f64         f64                f64           f64       ␣
↪ f64       f64

Sano             13925.0   38880.0     28787.0        …   1.0125        1198.0052␣
↪ 0.4837    8.2019
Cáncer           4855.0    7488.0      4988.0         …   -0.5605       279.8061 ␣
↪ 0.9733    1.2833
```

1 Github



1.1 Para entrar, de click aquí